DATA 430

PROJECT 2

Data Processing and Analytics with Apache Spark

Individual Sprint Project

Prepared by: Namita Utekar
Course: DATA 430
Submission date: November 5, 2026

Build an orchestrated pipeline that cleans source data, produces Spark analytics, and loads validated results into PostgreSQL.

1. Executive Summary

This project extends the earlier data pipeline work by adding Apache Spark-based data processing and analytical transformations, orchestrated through Apache Airflow and persisted in PostgreSQL. The implementation uses containerized services to run the processing and database components in a repeatable local environment.

The completed workflow validates source files, executes cleansing, generates analytical outputs, loads cleaned and analytical datasets into PostgreSQL, validates database results, and marks the workflow complete. The final Airflow run on November 5, 2026 completed successfully across all six configured tasks.

The database contains three core entity tables (customers, products, and orders), the ETL audit table, and twelve analytics tables. Verified counts include 99,900 customers, 99,861 products, 98,457 orders, and all twelve expected analytics outputs.

1.1 Project outcomes

Containerized execution environment configured for Airflow, Spark, and PostgreSQL.

Source datasets generated with 100,000 records per entity.

Spark cleansing outputs persisted as Parquet for customers, products, and orders.

Twelve analytical datasets generated and loaded into PostgreSQL.

All six tasks in the final Airflow DAG run completed successfully.

Database validation confirmed expected counts across fifteen core and analytics tables.

ETL audit history retained successful loads and an earlier failed attempt.

1.2 Scope and evidence

This report documents the implementation and results verified during project work. Requirements not demonstrated by the available code or outputs are identified as limitations or future improvements rather than claimed as completed.

2. Solution Architecture and Environment

The solution is organized into four layers: source data, Spark processing, orchestration, and relational storage. Airflow coordinates the workflow, Spark performs data preparation and analytics, and PostgreSQL stores resulting datasets and ETL audit records.

Component	Role
Docker Compose	Runs project services in isolated containers.
Apache Airflow	Orchestrates the six-task Project 2 DAG.
Apache Spark	Processes source data, writes cleaned Parquet data, and creates analytics outputs.
PostgreSQL	Stores core entities, audit records, and twelve analytics tables.
Python scripts	Implement cleansing, analytics, and database loading.

2.1 Local service endpoints

Service	Container / endpoint
Airflow	data430_project2_airflow — http://localhost:8081
PostgreSQL	data430_project2_postgres — host port 5433
Spark Master	data430_project2_spark_master — http://localhost:8082
Spark Worker	data430_project2_spark_worker — host port 8083

The PostgreSQL database is named data430. Credentials are stored in the local .env file and are intentionally not reproduced in this report.

2.2 Project organization

The project directory is DATA430_Project2. It includes Docker Compose configuration, Python requirements, data and scripts directories, SQL schemas, and an Airflow DAG. Principal components include spark_utils.py, spark_cleanse.py, spark_analytics.py, load_to_postgres.py, and dags/data430_project2_pipeline.py.

3. Source Data and Data Preparation

The project uses three related business entities: customers, products, and orders. The improved data-generation run produced 100,000 records for each CSV dataset. The generated data was designed to exercise data-quality handling.

Entity	Generated records	Final database rows
Customers	100,000	99,900
Products	100,000	99,861
Orders	100,000	98,457

The difference between generated and final row counts reflects processing and loading outcomes. The exact number attributable to each individual cleansing rule was not independently captured in the evidence summarized here; the differences should not be interpreted as per-rule rejection counts.

3.1 Cleansing workflow

The cleansing stage is implemented in spark_cleanse.py and supported by spark_utils.py. It processes customer, product, and order data and writes cleaned results in Parquet format for subsequent analytics and loading.

Validate that required source files are available.

Apply the project cleansing transformations to customer, product, and order records.

Write cleaned entity datasets to Parquet output folders.

Make the cleaned outputs available to analytics and database-loading stages.

3.2 Data-quality considerations

The project scenario includes duplicate identifiers, missing values, inconsistent casing and whitespace, invalid numeric values, inconsistent date formats, and orders that may not reference a valid customer. The loader was updated to normalize customer-type values before loading to PostgreSQL. This resolved the earlier case-sensitive check-constraint failure caused by the value 'Vip'.

The final Airflow validation checks for orphan order-to-customer references. The successful final run indicates that this configured validation passed.

4. Spark Analytics

The Spark analytics stage is implemented in spark_analytics.py. It produces twelve analytical datasets, which are written to the analytics output location and loaded into corresponding PostgreSQL tables.

Analytics output	Rows	Purpose
customer_metrics	62,580	Customer-level lifetime value, order frequency, average order value, and customer type.
top_10_customers	10	Top ten customers by the implemented customer revenue metric.

Analytics output	Rows	Purpose
customer_type_metrics	3	Summary measures grouped by customer type.
category_metrics	8	Category counts, average price, stock, and active-product measures.
inventory_metrics	99,861	Product-level inventory and descriptive attributes.
monthly_sales	24	Sales and order measures by month.
daily_sales	730	Sales and order measures by date.
order_status_metrics	3	Order counts and sales grouped by order status.
payment_metrics	5	Order counts and sales grouped by payment method.
discount_metrics	1	Overall discount, shipping-cost, and sales summary.
revenue_by_customer_type	3	Revenue and order counts grouped by customer type.
revenue_by_city	29,103	Revenue and order counts grouped by city.

4.1 Analytical interpretation

The outputs provide complementary views of customer value, product inventory, sales trends, order status, payment behavior, discounts, and geographic revenue. Customer metrics support customer-value analysis; monthly and daily sales support time-based analysis; and status and payment outputs summarize operational patterns.

4.2 Important analytical limitation

The available orders schema does not include product_id or order-line quantity fields. Therefore, the current model cannot calculate true product-level sales revenue or product popularity from order lines, nor category revenue by customer type. Category and inventory outputs describe product catalog and stock characteristics rather than product sales performance.

5. Airflow Orchestration

The Airflow workflow is defined in dags/data430_project2_pipeline.py. The configured DAG contains six sequential tasks. The final run was manually triggered on November 5, 2026 and completed successfully.

Sequence	Task	Function
1	file_validation	Checks required input files.
2	spark_cleanse	Runs the data cleansing stage.
3	spark_analytics	Generates twelve analytical outputs.
4	load_to_postgres	Loads cleaned entity data and analytics results into PostgreSQL.
5	data_validation	Checks expected table counts and orphan customer references.
6	completion	Marks the workflow complete after preceding tasks succeed.

5.1 Final execution result

Task	Final state
file_validation	success
spark_cleanse	success
spark_analytics	success
load_to_postgres	success
data_validation	success
completion	success

The final Airflow graph screenshot supplied during project work shows all six tasks in the success state. This is the primary evidence that end-to-end orchestration completed.

5.2 Operational notes

The local environment uses a containerized Airflow service and a separate PostgreSQL data store. Non-fatal Spark warnings were observed, including a missing ps utility warning and a native Hadoop library warning. They did not prevent processing or the final pipeline run.

6. PostgreSQL Storage and Loading

PostgreSQL stores cleaned core datasets, ETL audit history, and twelve analytical outputs. The original schema contains customers, products, orders, and etl_audit. A separate analytics_schema.sql defines the twelve analytics tables.

Database table	Verified rows
customers	99,900
products	99,861
orders	98,457
customer_metrics	62,580
top_10_customers	10
customer_type_metrics	3
category_metrics	8
inventory_metrics	99,861
monthly_sales	24
daily_sales	730
order_status_metrics	3
payment_metrics	5
discount_metrics	1
revenue_by_customer_type	3
revenue_by_city	29,103

6.1 Loading approach

The load_to_postgres.py script preserves core customer, product, and order loading and loads all twelve analytics outputs from the analytics output directory. Analytics tables are refreshed using a transactional delete-and-insert pattern, replacing the current analytics result set rather than appending duplicate rows. The loader records SUCCESS or FAILED entries in etl_audit.

6.2 Database validation

A direct PostgreSQL query confirmed the expected row count in each of the fifteen core and analytics tables. The Airflow data_validation task checks expected counts for these fifteen tables and checks for orphan order customer references. The final Airflow run reported success for this validation task.

7. Testing, Validation, and Audit Results

7.1 End-to-end validation

The final Airflow run completed all six tasks successfully. The database verification query returned fifteen table counts matching expected values. Together, these checks provide evidence that final orchestration, loading, and configured database validation completed.

7.2 ETL audit history

Audit status	Entries	Records processed
SUCCESS	36	1,577,534
FAILED	1	0

At the time of inspection, the audit table contained 36 successful entries and one failed entry. The failed entry is audit_id 1 for customers, dated November 4, 2026. Its message reports a PostgreSQL check-constraint violation because customer_type value 'Vip' did not satisfy customers_customer_type_check.

This earlier failure occurred before the loader was updated to normalize customer-type values. The subsequent November 5 pipeline run completed successfully. The historical failed record was retained, preserving operational history.

7.3 Evidence captured

Airflow DAG graph showing all six tasks successful.

PostgreSQL query output showing all fifteen verified table counts.

ETL audit query output showing successful load records and the earlier failed record.

ETL audit summary showing 36 SUCCESS entries and one FAILED entry.

Attach the screenshots as separate evidence files or include them in an appendix if the course submission accepts supporting images.

8. Limitations and Future Improvements

The completed pipeline demonstrates orchestration, Spark processing, analytics generation, PostgreSQL loading, and database validation. The following improvements would strengthen the solution:

Add order-line detail containing product_id and quantity so product revenue, product popularity, and category sales performance can be calculated accurately.

Add category-by-customer-type revenue analysis once order-line and customer relationships support that level of detail.

Capture explicit per-rule cleansing statistics: input rows, output rows, duplicates removed, nulls handled, invalid records rejected, and orphan records removed.

Add automated unit tests for cleansing functions and analytics transformations.

Add a README evidence section with exact commands, expected outputs, and screenshots.

Consider a dedicated Spark submission task if the deployment environment requires distributed Spark execution for the full workload.

9. Conclusion

DATA 430 Project 2 extends the earlier data pipeline with Spark-based cleansing and analytics, Airflow orchestration, and PostgreSQL persistence. The final DAG execution completed all six tasks successfully. Fifteen database tables were verified against expected row counts, and the ETL audit table retained successful load history and an earlier failed attempt.

The implementation provides a working foundation for repeatable data preparation and business analytics. The principal analytical limitation is the absence of order-line product identifiers and quantities, which prevents true product-level sales analysis. With that limitation documented, the verified results demonstrate a successful end-to-end pipeline.

Appendix A. Key project files

File	Purpose
docker-compose.yml	Defines local containerized services.
requirements.txt	Lists Python dependencies.
scripts/spark_utils.py	Shared Spark utilities.
scripts/spark_cleanse.py	Cleans customer, product, and order datasets.
scripts/spark_analytics.py	Builds twelve analytical datasets.
scripts/load_to_postgres.py	Loads core and analytics data and records ETL audit entries.
dags/data430_project2_pipeline.py	Defines Airflow workflow and database validation.
sql/schema.sql	Defines core database tables.
sql/analytics_schema.sql	Defines twelve analytics tables.

Appendix B. Evidence checklist: include the successful Airflow DAG screenshot, the PostgreSQL fifteen-table count screenshot, and the ETL audit query/summary screenshots. Submit the project folder, source scripts, SQL schemas, and README if required by the instructor.